

DIGITAL SYSTEMS ELECTRONICS

Academic year 2023-2024

LABORATORY NR. 7

DUE DATE: May 20, 2024
DELIVERY DATE: May 20, 2024

GROUP 03

CONTRIBUTIONS:

- El Amrani Dawid 100%
- Ielmini Giovanni 100%
- Di Carolo Pierfelice 100%
- Pavone Lorenzo 100%

The members of the group listed above declare under their own responsibility that no part of this document has been copied from other documents and that the associated code is original and has been developed expressly for the assigned project.

Contents

This report is relative to the seventh laboratory experience of this course.
The purpose of this lab session was to learn how to drive the light emitting diode (LED) and to use the pushbutton available on the NUCLEO board

1. Program Example

First, as stated in the instructions, we created a new project on the IDE. Then we copied and pasted the example code and downloaded it on the Nucleo board.

As expected, the LED started blinking.

After verifying the correctness of the code, we tested the “debug” function. After setting a breakpoint in the loop, we launched the debug, and saw how it executes one by one the instructions on the code, as we “step into” them.

By looking at the values of the peripheral registers, we understood that the first three lines initialised the three registers involved.

When the program reaches the breakpoint the execution will stop before executing this line, at this point the LED will be ON because the value of 'GPIOA_ODR' at this moment will have bit 5 set to 1, so PA5 is high.

After the breakpoint, when the program executes '`*GPIOA_ODR = *GPIOA_ODR ^ 0x20`', bit 5 of 'GPIOA_ODR' will be toggled: if it was 1, it will become 0, and vice versa.

If PA5 was high (LED on), it will become low (LED off), and vice versa.

The variable `i` will then be used for a delay loop, and the process of toggling PA5 will continue in the infinite loop, making the LED connected to PA5 blink.

2. Using a switch to switch on/off the LED, Low Level Approach

In this exercise we modified the previous code so that by pressing the pushbutton the LED turns ON.

We know that the pushbutton is connected to PC13, that is bit 13 of GPIOC, so we initialize the GPIOC peripheral, setting it to the proper address, so that pin 13 is recognized as an input bit. Then we modify the AHB1 peripheral clock enable register setting the value at 0x05, that is the results from combining 0x01 (for GPIOA) and 0x04 (for GPIOC).

The if statement checks if the button connected to PC13 is pressed, 0x2000 corresponds to bit 13 (PC13), if the button is pressed the result of the bitwise AND will be 0, making the condition '!' true, so the LED is turned ON by setting bit 5 of the ODR (PA5) to 1, while, if the button is not pressed the LED is turned OFF by setting bit 5 of the ODR (PA5) to 0.

3. Using a switch to switch on/off the LED, STM32Cube approach

In this exercise, we had to solve the same problem as the previous one but using the LL developed by ST.

Firstly we activated the pin PC13 as an input (which corresponds to the button) and pin PA5 as an output (which corresponds to the LED2) using the graphic interface, then we generated the code. Starting from the generated code we modify the while statement adding an if statement, in particular: '`LL_GPIO_ReadReg(GPIOC, IDR)`' reads the value of the input data register (IDR) of GPIOC; '`& 0x2000`' performs a bitwise AND operation with the mask 0x2000 (which corresponds to

bit 13), this checks if bit 13 is set; '!' negates the result of the AND operation, therefore, this condition is true if bit 13 of the GPIOC IDR register is 0 (low).

If the if condition is true (bit 13 of GPIOC is low), then:

'LL_GPIO_ReadReg(GPIOA, ODR)' reads the value of the output data register (ODR) of GPIOA;

'| 0x20' performs a bitwise OR operation with the mask 0x20 (which corresponds to bit 5), this sets bit 5 to high; 'LL_GPIO_WriteReg(GPIOA, ODR, ...)' writes the new value (with bit 5 set) to the ODR register of GPIOA.

If the if condition is false:

'LL_GPIO_ReadReg(GPIOA, ODR)' reads the value of the ODR register of GPIOA;

'& ~(0x20)' performs a bitwise AND operation with the complement of the mask 0x20, this sets bit 5 to 0; 'LL_GPIO_WriteReg(GPIOA, ODR, ...)' writes the new value to the ODR register of GPIOA.

4. *Varying the blinking frequency*

In this exercise, we had to develop a C program that doubles the blinking frequency of the LED each time we press the button, starting from a frequency of 0.25 Hz.

Firstly we activated the pin PC13 as an input (which corresponds to the button) and pin PA5 as an output (which corresponds to the LED2). After having generated the code and having checked that the clock signal and peripherals were correctly set we wrote our code. Into the main, we set two integer variables: MYWAIT and pushed. MYWAIT is set to 3500000 as this is the value that in the final code corresponds to 0.25 Hz, pushed is instead set to 0 and is the value that tells us if the button has been pushed. In the while(1){} loop we have an if statement that when pushed is equal to 1 sets MYWAIT to half its previous value and "pushed" again to 0. Then we write into the LED's register with the LL_GPIO_WriteReg function using an XOR and a mask (0x20) to change the bit in position 5 from a 0->1 or 1->0 depending on its previous value (this makes the LED blink):

```
LL_GPIO_WriteReg( GPIOA, ODR, (LL_GPIO_ReadReg( GPIOA, ODR) ^ 0x20)
);
```

Then we have a for cycle that counts until we reach the value MYWAIT. Inside of it, we have an if statement that when the button is pushed sets pushed to 1. This allows us to read the value of the button each cycle. To read it we use the LL_GPIO_ReadReg function with the mask for 0x2000 that corresponds to position 13 (the button):

```
LL_GPIO_ReadReg( GPIOC, IDR) & 0X2000
```

We negate the reading as the button is an active-low.

When we reach a frequency of around 16 Hz the flashing light appears continuously on. That's because the limit for our eye in order to see a light flashing is around 18 Hz.

5. *Generating a Square Wave*

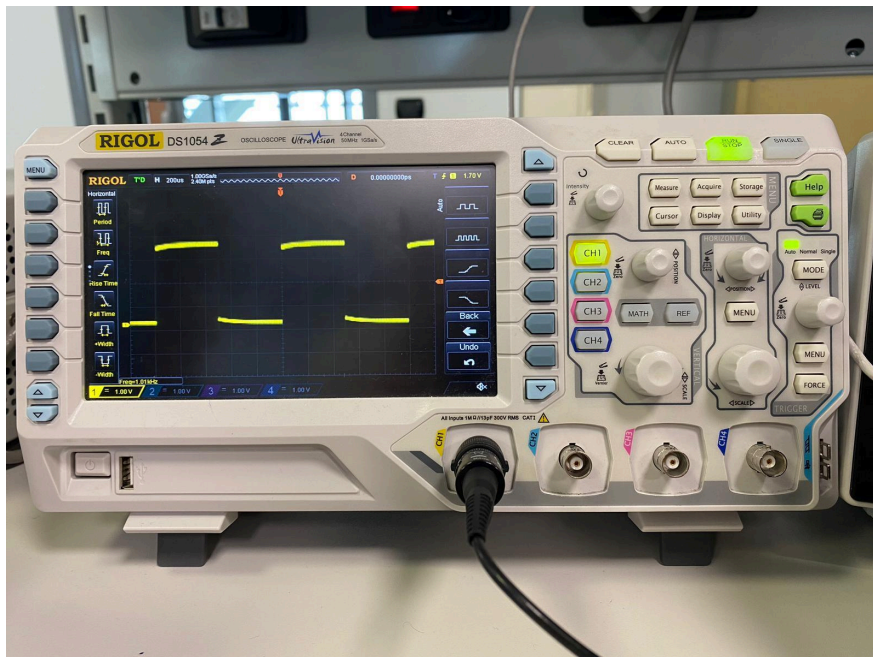
For this design we had to generate a square wave using pin D2 at a frequency of 1 kHz. To do that we activated PA10 as output. Then we added the pieces of code required by the text both in the

“main.c” and in “stm32f4xx_it.c”. To activate and deactivate the pin in order to generate a square wave we used the same approach of the previous exercise. We defined the value of “MYWAIT” at 3000, which we found was the value for a 1 kHz square wave. Then we just used the following piece of code into the infinite loop:

```
LL_GPIO_WriteReg( GPIOA, ODR, (LL_GPIO_ReadReg(GPIOA,ODR) ^ 0x400) );  
    for (int i=0; i<MYWAIT; i++){ }
```

With the first line we write in the register GPIOA position 10 which corresponds to our output pin using a XOR (just like in the previous exercise) and a mask having a one in position 10 and all others positions as zero. This allows us to change every time the value of the bit in position 10. Then after that the “for” counts up to “MYWAIT”. As this is in the infinite loop it keeps repeating generating a square wave.

The following picture shows the outcome on the oscilloscope.



Modifying the code as requested makes the wave more precise.